Django Beginner to Advanced — Complete Learning Roadmap

Project: `live-portfolio`

Language: Python

Framework: Django

Database: SQLite → PostgreSQL

API: Django REST Framework

Level: Beginner → Intermediate → Advanced → Professional

Table of Contents

Level 1 — Beginner

1. Python Prerequisites
2. Django Introduction
3. Environment Setup
4. Django Project and App
5. URL Routing
6. Views
7. Templates
8. Static Files
9. Models and Database
10. Django ORM Basics
11. Django Admin
12. Forms
13. CRUD Operations
14. Mini Project

Level 2 — Intermediate

15. Model Relationships
16. Advanced QuerySets
17. Custom Managers
18. Authentication
19. Authorization and Permissions

20. Sessions and Cookies
21. Messages Framework
22. Pagination
23. File Uploads
24. Class-Based Views
25. Middleware
26. Signals
27. Django Testing
28. PostgreSQL
29. Performance Optimization
30. Intermediate Project

Level 3 — Advanced

31. Advanced ORM
32. Transactions
33. Query Optimization
34. Caching
35. Security
36. Custom User Model
37. Django REST Framework
38. Authentication for APIs
39. Permissions and Throttling
40. Filtering and Pagination for APIs
41. API Testing
42. Background Tasks
43. Logging and Monitoring
44. Advanced Project

Level 4 — Professional

45. Production Settings
46. Environment Variables and Secrets
47. HTTPS and Security Headers
48. Static and Media Storage
49. PostgreSQL in Production
50. WSGI and ASGI

51. Gunicorn and Reverse Proxy
52. Docker Fundamentals
53. CI/CD
54. Automated Testing
55. Deployment
56. Monitoring and Backups
57. Final Production Project
58. Interview Preparation

Level 1 — Beginner

1. Python Prerequisites

Django শেখার আগে Python-এর এই বিষয়গুলো জানতে হবে।

- Variables and data types
- Strings and numbers
- Lists, tuples, sets and dictionaries
- Conditions
- Loops
- Functions
- Modules and packages
- Exception handling
- File handling
- List comprehensions
- Object-oriented programming
- Classes and objects
- Inheritance
- Decorators
- Virtual environments
- `pip` and dependency management

Example:

```
def calculate_total(price, quantity):
    return price * quantity
```

```
total = calculate_total(100, 3)
print(total)
```

Expected output:

```
300
```

2. What Is Django?

Django একটি Python web framework। এটি database-driven website এবং web application তৈরি করতে সাহায্য করে।

Django-এর গুরুত্বপূর্ণ বৈশিষ্ট্য:

- URL routing
- Views and templates
- ORM
- Authentication
- Admin panel
- Forms and validation
- Security features
- Middleware
- Testing tools

Django request flow:

```
Browser
   |
   v
URL Configuration
   |
   v
View
   |
   v
Model / Database
   |
   v
Template
   |
   v
HTTP Response
```

3. Environment Setup

Virtual environment তৈরি করো:

```
python -m venv .venv
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Windows CMD:

```
.venv\Scripts\activate.bat
```

Django install:

```
python -m pip install django
```

Version check:

```
python -m django --version
```

4. Create a Django Project

```
django-admin startproject config .
```

Development server:

```
python manage.py runserver
```

Browser:

```
http://127.0.0.1:8000/
```

Important files:

```
project/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
└── .venv/
```

5. Create a Django App

```
python manage.py startapp products
```

Add it to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Django applications
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Local applications
    "products",
]
```

একটি project-এ একাধিক app থাকতে পারে। প্রতিটি app একটি নির্দিষ্ট functionality পরিচালনা করতে পারে।

6. URL Routing

`products/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.product_list, name="product_list"),
]
```

Project-এর `config/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("products/", include("products.urls")),
]
```

7. Views

`products/views.py`:

```
from django.http import HttpResponse
```

```
def product_list(request):
    return HttpResponse("Product List")
```

View একটি HTTP request গ্রহণ করে এবং একটি HTTP response তৈরি করে।

8. Templates

`products/views.py`:

```
from django.shortcuts import render


def product_list(request):
    products = ["Laptop", "Phone", "Keyboard"]

    return render(
        request,
        "products/product_list.html",
        {"products": products},
    )
```

Template file:
`products/templates/products/product_list.html`

```
<h1>Product List</h1>

<ul>
    {% for product in products %}
        <li>{{ product }}</li>
    {% empty %}
        <li>No products found.</li>
    {% endfor %}
</ul>
```

Templates-এ শিখবে:

- Variables
- `{% for %}`
- `{% if %}`
- Template inheritance
- `{% extends %}`
- `{% block %}`
- `{% include %}`
- Automatic HTML escaping

9. Static Files

Static files হলো CSS, JavaScript এবং application-এর নিজস্ব image files।

`settings.py`:

```
STATIC_URL = "static/"
```

Template:

```
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Development-এ static file handling এবং production-এ static file serving এক নয়।

10. Models and Database

`products/models.py`:

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=150)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
    description = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Common model fields:

- `CharField`
- `TextField`
- `IntegerField`
- `DecimalField`
- `BooleanField`
- `DateField`

- `DateTimeField`
- `EmailField`
- `FileField`
- `ImageField`
- `ForeignKey`
- `OneToOneField`
- `ManyToManyField`

11. Django Admin

`products/admin.py`:

```
from django.contrib import admin
from .models import Product


@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ("name", "price", "created_at")
    search_fields = ("name",)
    list_filter = ("created_at",)
```

Create an admin user:

```
python manage.py createsuperuser
```

Open:

```
http://127.0.0.1:8000/admin/
```

12. Django ORM Basics

Create:

```
Product.objects.create(
    name="Keyboard",
    price=1500,
)
```

Read:

```
Product.objects.all()
Product.objects.filter(price__gt=1000)
Product.objects.get(pk=1)
```

Update:

```
Product.objects.filter(pk=1).update(price=2000)
```

Delete:

```
Product.objects.filter(pk=1).delete()
```

Remember:

- `filter()` returns a QuerySet.
- `get()` expects exactly one matching object.
- `create()` creates and saves an object.
- `update()` performs a direct SQL update.
- `delete()` deletes matching records.

13. Forms and Validation

`products/forms.py`:

```
from django import forms
from .models import Product


class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "price", "description"]
```

A form handles input, validation and cleaned data.

For POST forms, use CSRF protection:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>
```

14. Beginner Mini Project

Project: Product Management System

Features:

- Product list
- Product detail
- Add product
- Edit product

- Delete product
- Search
- Admin management
- Form validation

Beginner level শেষ করার আগে এই project নিজে তৈরি করার চেষ্টা করো।

Level 2 — Intermediate

15. Model Relationships

ForeignKey

Many products belong to one category.

```
class Category(models.Model):
    name = models.CharField(max_length=100)


class Product(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.CASCADE,
        related_name="products",
    )
    name = models.CharField(max_length=150)
```

OneToOneField

One profile belongs to one user.

```
from django.conf import settings


class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )
    bio = models.TextField(blank=True)
```

ManyToManyField

Many students can enroll in many courses.

```
class Course(models.Model):
    title = models.CharField(max_length=100)


class Student(models.Model):
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(Course)
```

16. Advanced QuerySet Operations

```
Product.objects.order_by("-price")

Product.objects.values("name", "price")

Product.objects.values_list("name", flat=True)

Product.objects.exclude(price__lt=100)

Product.objects.filter(name__icontains="phone")

Product.objects.filter(price__range=(100, 1000))

Product.objects.exists()

Product.objects.count()
```

Date queries:

```
Product.objects.filter(created_at__year=2026)
```

17. Custom Managers and QuerySets

```
from django.db import models


class ProductQuerySet(models.QuerySet):
    def expensive(self):
        return self.filter(price__gt=5000)

    def affordable(self):
        return self.filter(price__lte=5000)


class Product(models.Model):
    name = models.CharField(max_length=100)
```

```
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    objects = ProductQuerySet.as_manager()
```

Usage:

```
Product.objects.expensive()
Product.objects.filter(name__icontains="phone").expensive()
```

18. Authentication

Django provides authentication features such as:

- Login
- Logout
- Password hashing
- Password reset
- User permissions
- Groups

Use Django's authentication framework rather than implementing password storage from scratch.

19. Authorization and Permissions

Authentication means identifying the user.

Authorization means deciding what the user can do.

Examples:

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render


@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

For protected operations, also check ownership and permissions. Being logged in does not automatically authorize a user to modify every record.

20. Sessions and Cookies

Session configuration:

```
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE = True
```

Enable secure cookies when the application is correctly served over HTTPS.

Learn:

- Session creation
- Session expiration
- Login session management
- Cookie attributes
- Session fixation prevention
- Logout behavior

21. Messages Framework

```
from django.contrib import messages
from django.shortcuts import redirect


def save_product(request):
    messages.success(request, "Product saved successfully.")
    return redirect("product_list")
```

Template:

```
{% for message in messages %}
    <p>{{ message }}</p>
{% endfor %}
```

22. Pagination

```
from django.core.paginator import Paginator
from django.shortcuts import render


def product_list(request):
    products = Product.objects.order_by("pk")
    paginator = Paginator(products, 10)

    page_number = request.GET.get("page")
    page_obj = paginator.get_page(page_number)

    return render(
        request,
```

```
        "products/product_list.html",
        {"page_obj": page_obj},
    )
```

Pagination helps manage large result sets.

23. File Uploads

Model:

```
class Document(models.Model):
    title = models.CharField(max_length=100)
    file = models.FileField(upload_to="documents/")
```

Learn:

- `request.FILES`
- Multipart form encoding
- File validation
- File size limits
- Media storage
- Safe file serving
- User authorization

Do not rely only on the filename extension to determine whether an uploaded file is safe.

24. Class-Based Views

Example:

```
from django.views.generic import ListView
from .models import Product


class ProductListView(ListView):
    model = Product
    template_name = "products/product_list.html"
    context_object_name = "products"
    paginate_by = 10
```

Learn:

- `View`
- `TemplateView`
- `ListView`
- `DetailView`
- `CreateView`

- `UpdateView`
- `DeleteView`
- Mixins
- Permission checks

25. Middleware

Middleware processes requests and responses around view execution.

Examples:

- Security middleware
- Session middleware
- CSRF middleware
- Authentication middleware
- Custom middleware

Learn the middleware lifecycle, ordering and how to avoid unnecessary processing.

26. Signals

Signals allow components to respond to certain events.

Examples include `post_save` and `pre_delete`.

Use signals carefully. Important business logic is often easier to understand when placed in explicit service functions or model methods rather than hidden across multiple signals.

27. Django Testing

Example:

```
from django.test import TestCase
from .models import Product


class ProductModelTests(TestCase):
    def test_product_creation(self):
        product = Product.objects.create(
            name="Mouse",
            price=500,
        )

        self.assertEqual(product.name, "Mouse")
```

Learn:

- Unit tests
- Model tests

- View tests
- Form tests
- Client requests
- Authentication tests
- Permission tests
- Test database setup

28. PostgreSQL

Learn:

- Database connection settings
- Database users and permissions
- Migrations
- Constraints and indexes
- Transactions
- Backups and restores
- Query plans
- Connection configuration

For a production backend, PostgreSQL is a valuable database to learn.

29. Performance Optimization

Use:

```
Product.objects.select_related("category")
```

for suitable ForeignKey or OneToOne relationships.

Use:

```
Category.objects.prefetch_related("products")
```

for reverse ForeignKey or ManyToMany relationships.

Also learn:

- Database indexes
- Pagination
- QuerySet evaluation
- Avoiding N+1 queries
- `QuerySet.explain()`
- Caching
- Profiling

30. Intermediate Project

Project: Blog Management System

Features:

- Registration and login
- User profiles
- Blog CRUD
- Categories and tags
- Comments
- Search and pagination
- Permissions
- Image uploads
- Tests
- PostgreSQL

Level 3 — Advanced

31. Advanced ORM

Study:

- `Q` objects
- `F` expressions
- `Case` and `When`
- `Subquery`
- `OuterRef`
- `Exists`
- `select_related`
- `prefetch_related`
- `Prefetch`
- `annotate`
- `aggregate`
- Window functions

Example:

```
from django.db.models import F

Product.objects.filter(
    stock__lt=F("reorder_level")
)
```

`F()` expressions allow comparisons or updates involving database fields without first retrieving their values into Python.

32. Transactions

```
from django.db import transaction


with transaction.atomic():
    # Perform related database operations.
    ...
```

Transactions help ensure that related database changes succeed or fail together.

Learn:

- Atomic transactions
- Rollbacks
- Integrity errors
- Database isolation concepts
- Concurrency
- `select_for_update()` where supported
- Transaction boundaries

33. Query Optimization

Learn how to inspect SQL and query plans.

```
queryset = Product.objects.filter(price__gt=1000)

print(queryset.explain())
```

Optimize only after identifying the actual bottleneck.

Important concepts:

- Indexes
- Joins
- Query plans
- N+1 queries
- Database constraints
- Efficient pagination
- Avoiding unnecessary data loading

34. Caching

Django supports several caching approaches.

Learn:

- Per-site cache
- Per-view cache
- Template fragment cache
- Low-level cache API
- Cache invalidation
- Redis-backed caching
- Cache key design

Example:

```
from django.core.cache import cache

cache.set("homepage_title", "Welcome", timeout=300)

title = cache.get("homepage_title")
```

For multi-user or sensitive data, cache keys and invalidation must be designed carefully to avoid leaking one user's data to another.

35. Advanced Security

Study:

- XSS
- CSRF
- SQL injection
- Clickjacking
- Host header validation
- Session security
- Password reset security
- File upload security
- Rate limiting
- Authorization
- Dependency vulnerabilities
- Secret management
- Security headers

Run:

```
python manage.py check --deploy
```

This command detects certain deployment configuration issues but is not a complete security audit.

36. Custom User Model

For a new project that needs a custom user model, consider defining it before the first migrations.

Example outline:

```
from django.contrib.auth.models import AbstractUser


class User(AbstractUser):
    pass
```

Then configure:

```
AUTH_USER_MODEL = "accounts.User"
```

Replace `accounts` with your actual app label.

Changing the user model late in an existing project can require complex database migrations.

37. Django REST Framework

Install:

```
pip install djangorestframework
```

Add to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Existing applications
    "rest_framework",
]
```

Learn:

- Serializers
- ModelSerializer
- API views
- Generic views
- ViewSets

- Routers
- Request and Response
- Validation
- HTTP status codes
- API versioning

Example serializer:

```
from rest_framework import serializers
from .models import Product


class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price"]
```

38. API Authentication

Study the differences between:

- Session authentication
- Token authentication
- JWT authentication
- OAuth 2.0
- API keys and service credentials

These mechanisms are not interchangeable. Choose according to the client, threat model and system requirements.

39. API Permissions and Throttling

Example:

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.views import APIView


class PrivateAPIView(APIView):
    permission_classes = [IsAuthenticated]
```

Learn:

- `AllowAny`
- `IsAuthenticated`
- `IsAdminUser`

- Object-level permissions
- Custom permission classes
- Throttling
- Ownership checks
- Pagination
- Filtering

Throttling can reduce abusive traffic but should not be treated as complete denial-of-service protection.

40. API Testing

Test:

- Successful requests
- Invalid input
- Authentication
- Permissions
- Object ownership
- Pagination
- Filtering
- Status codes
- Database behavior
- Error responses

Use Django and Django REST Framework's test tools.

41. Background Tasks

Long-running work may need to execute outside the request-response cycle.

Learn:

- Background task queues
- Celery
- Redis or another supported broker
- Retries
- Timeouts
- Idempotency
- Task monitoring

Example use cases:

- Sending emails

- Processing large files
- Generating reports
- Scheduled maintenance

42. Logging and Monitoring

Learn:

- Python logging
- Django logging configuration
- Error reporting
- Application metrics
- Health checks
- Request tracing
- Database monitoring
- Safe log redaction

Never log passwords, authentication tokens or secret keys.

43. Advanced Project

Project: E-commerce REST API

Features:

- Custom user model
- Product and category management
- Product search and filtering
- Cart and orders
- Permissions
- Payment-provider integration
- PostgreSQL
- API documentation
- Automated tests
- Caching
- Background tasks
- Logging and monitoring

For payments, use a reputable payment provider and follow its security requirements rather than storing sensitive payment credentials yourself.

Level 4 — Professional

44. Production Settings

Separate local development configuration from production configuration.

Production basics:

```
DEBUG = False

ALLOWED_HOSTS = [
    "yourdomain.com",
]
```

Use environment variables or an appropriate secret manager for sensitive configuration.

45. Environment Variables and Secrets

Generate a Django key:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

Read it in Django:

```
import os

SECRET_KEY = os.environ["SECRET_KEY"]
```

Never commit production secrets to source control.

46. HTTPS and Security Headers

After HTTPS and proxy configuration have been verified:

```
SECURE_SSL_REDIRECT = True

SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = "DENY"
```

Configure HSTS only after confirming HTTPS works correctly:

```
SECURE_HSTS_SECONDS = 86400
```

Increase the duration only when appropriate for the domain and all affected subdomains.

47. Static and Media Storage

Learn:

- `STATIC_URL`
- `STATIC_ROOT`
- `collectstatic`
- Static storage backends
- `MEDIA_URL`
- `MEDIA_ROOT`
- Object storage
- CDN configuration
- Private file downloads

Static files and user-uploaded media have different security and operational requirements.

48. PostgreSQL in Production

Learn:

- Production database credentials
- Least-privilege database roles
- Network access controls
- TLS where appropriate
- Connection limits
- Backups
- Restore testing
- Migration strategy
- Index monitoring

49. WSGI and ASGI

WSGI is the traditional Python web server interface.

ASGI supports asynchronous application patterns and long-lived connections.

Learn how your selected Django deployment stack uses either interface.

50. Gunicorn and Reverse Proxy

A common deployment architecture is:

```
Browser
   |
   v
HTTPS / Reverse Proxy
   |
   v
```

```
Gunicorn or compatible application server
   |
   v
Django Application
   |
   v
PostgreSQL
```

The actual architecture depends on the hosting platform.

Learn:

- Application server configuration
- Worker settings
- Reverse proxy headers
- Timeouts
- Static-file serving
- Process management
- Health checks

51. Docker Fundamentals

Learn:

- Docker images
- Containers
- Dockerfile
- Compose
- Environment variables
- Container networking
- Persistent storage
- Image security
- Non-root containers
- Secret handling

Docker is useful but is not mandatory for every Django deployment.

52. CI/CD

Learn how to automate:

- Code formatting
- Linting
- Unit tests

- Integration tests
- Dependency checks
- Build steps
- Deployment
- Rollbacks

Keep production secrets in the CI/CD platform's protected secret storage.

53. Automated Testing

A professional backend should test:

- Business logic
- Models
- Views
- APIs
- Authentication
- Authorization
- Security-sensitive behavior
- Database constraints
- Critical integrations

Tests should verify both expected success and failure cases.

54. Deployment

Generic workflow:

```
1. Review code
2. Install locked dependencies
3. Run automated tests
4. Configure environment variables
5. Prepare database
6. Apply migrations
7. Collect static files
8. Deploy application
9. Verify HTTPS
10. Run smoke tests
11. Monitor logs and errors
12. Confirm backup and rollback readiness
```

Follow the hosting platform's deployment instructions. Do not assume every platform requires the same commands or server architecture.

55. Monitoring and Backups

Learn:

- Structured logs
- Application error monitoring
- Uptime monitoring
- Database backup schedules
- Restore testing
- Alerting
- Incident response
- Dependency updates
- Rollback procedures

A backup is only useful if restoration has been tested.

56. Final Production Project

Project: Production-Ready Portfolio and Blog Platform

Recommended features:

- User authentication
- Admin management
- Portfolio projects
- Blog and categories
- Search and pagination
- Contact form
- REST API
- PostgreSQL
- Secure media uploads
- Automated tests
- Logging
- HTTPS
- Production environment variables
- Static file configuration
- Database backups
- CI/CD
- Monitoring

Use your existing `live-portfolio` project to implement these concepts incrementally.

Interview Preparation

Python

- What is OOP?
- What is inheritance?
- What is a decorator?
- What is a virtual environment?
- What is the difference between `self` and `cls`?
- What are exceptions and context managers?

Django Fundamentals

- What is Django MVT?
- What is the difference between a project and an app?
- What is a model?
- What are migrations?
- What is middleware?
- What is the request-response lifecycle?
- What is a Django Manager?
- What is a QuerySet?

ORM

- `filter()` vs `get()`
- `select_related()` vs `prefetch_related()`
- `aggregate()` vs `annotate()`
- What is a custom Manager?
- What are `Q` and `F` expressions?
- What is a database transaction?
- How do indexes improve queries?

Security

- What is XSS?
- What is CSRF?
- What is SQL injection?
- Why use `DEBUG=False`?
- What is `ALLOWED_HOSTS`?
- What are secure cookies?

- What is HSTS?
- How do you secure uploaded files?

API Development

- What is a serializer?
- What is a ViewSet?
- What are HTTP status codes?
- What is authentication vs authorization?
- What is object-level permission?
- How do you validate request data?
- How do you test an API?

Deployment

- What is HTTPS?
- What is WSGI vs ASGI?
- What does `collectstatic` do?
- Why use PostgreSQL in production?
- What is Docker?
- How do you manage secrets?
- How do you back up and restore a database?

Official Documentation

- Django documentation: https://docs.djangoproject.com/en/stable/
- Django tutorial: https://docs.djangoproject.com/en/stable/intro/tutorial01/
- Django ORM: https://docs.djangoproject.com/en/stable/topics/db/queries/
- Django Managers: https://docs.djangoproject.com/en/stable/topics/db/managers/
- Django Security: https://docs.djangoproject.com/en/stable/topics/security/
- Deployment checklist: https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
- Django REST Framework: https://www.django-rest-framework.org/

Final Learning Strategy

Follow this order:

```
Python Fundamentals
        |
        v
Django Basics
        |
        v
Models + ORM + CRUD
        |
        v
Relationships + Forms + Authentication
        |
        v
Testing + PostgreSQL
        |
        v
Advanced ORM + Performance
        |
        v
Django Security
        |
        v
Django REST Framework
        |
        v
Production Deployment
        |
        v
Professional Backend Projects
```

Golden Rule: Learn one topic, implement it in your project, write tests for it, and then move to the next topic.

Do not try to master every advanced feature before you can independently build a complete CRUD application.